Data Structures

Mushtari Sadia

Linked List

Different Types of Linked Lists

Category 1	Category 2	Category 3
Non-Dummy Headed	Singly	Linear
Dummy Headed	Doubly	Circular

Non Dummy Headed

$$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$$

First node contains value 10.

Dummy Headed

$$DH \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40$$

In this linked list, DH refers dummy head which is a node without any element. The first item is stored after the dummy head.

Singly

$$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$$

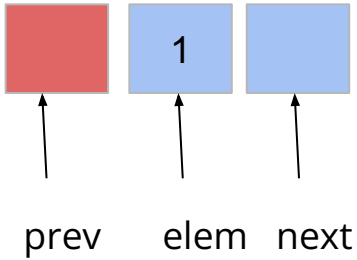
Only contains next pointer, no previous pointer

Doubly

10 ⇄ 20 ⇄ 30 ⇄ 40

Contains both next and previous pointer

```
class Node:
    def __init__(self, e, n, p):
        self.elem = e
        self.next = n
        self.prev = p
```



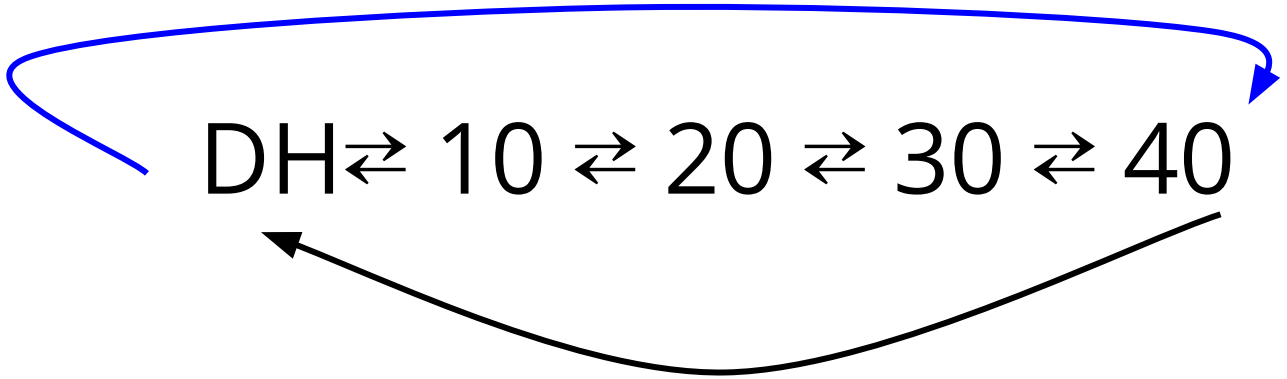
Linear vs Circular

Linear: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

Circular:

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

Dummy-headed doubly circular linked list



Dummy-headed doubly circular linked list

```
class DoublyNode:
        def __init__(self, elem, next, prev):
            self.elem = elem
            self.next = next # To store the next node's reference.
            self.prev = prev # To store the previous node's reference.
```

Dummy-headed doubly circular linked list

```
1  def createList(a):
2      dh = DoublyNode(None, None, None)
3      dh.next = dh
4      dh.prev = dh
5      tail = dh
6
7      for i in range(len(a)):
8          n = DoublyNode(a[i], dh, tail)
9          tail.next = n
10         tail = tail.next
11         dh.prev = tail
12
13     return dh
```

Dummy-headed doubly circular linked list

```
1 def iteration(dh):
2     temp = dh.next
3     while temp != dh:
4         print(temp.elem)
5         temp = temp.next
```

Dummy-headed doubly circular linked list

```
1 def nodeAt(dh, idx):
2   temp = dh.next
3   c = 0
4   while temp != dh:
5     if c == idx:
6       return temp
7     c += 1
8     temp = temp.next
9   return None # Invalid Index
```

Dummy-headed doubly circular linked list

```
1 def insertion(dh, elem, idx):
2   # Assuming the idx is valid
3   node_to_insert = DoublyNode(elem, None, None)
4   indexed_node = nodeAt(dh, idx) # Retriving the node at that index
5   prev_node = indexed_node.prev # There will always be a previous node
6   # Change the connection
7   # Observe that no special case is needed
8   node_to_insert.next = indexed_node
9   node_to_insert.prev = prev_node
10  prev_node.next = node_to_insert
11  indexed_node.prev = node_to_insert
```

Dummy-headed doubly circular linked list

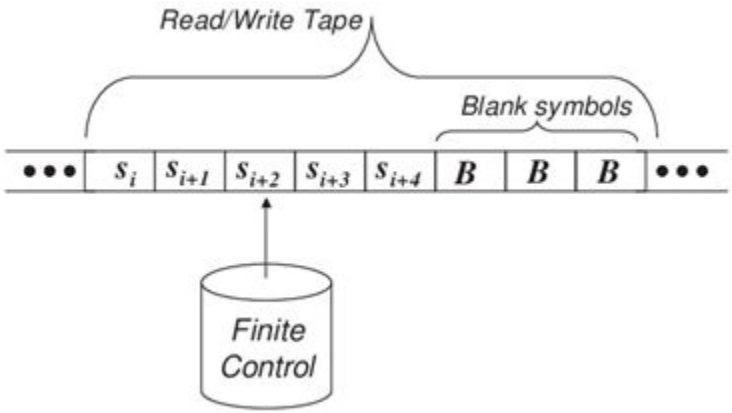
```
1 def removal(dh, idx):
2   # Assuming the idx is valid
3   node_to_remove = nodeAt(dh, idx)
4   prev_node = node_to_remove.prev
5   next_node = node_to_remove.next
6   # Change the connection
7   # No special case is needed
8   prev_node.next = next_node
9   next_node.prev = prev_node
10  node_to_remove.next = None
11  node_to_remove.prev = None
12  return node_to_remove.elem # Returning the removed element
```

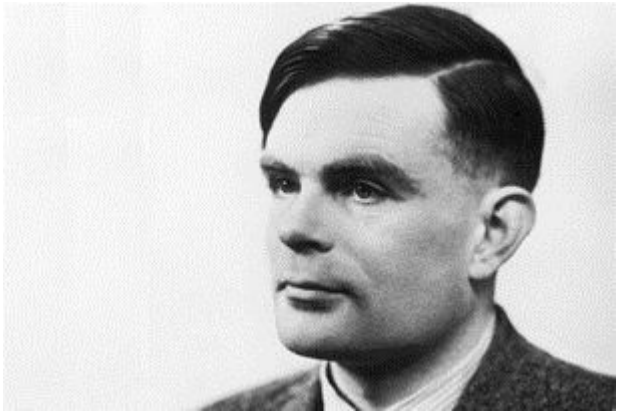
Reasons for doubly linked list

Problem of singly linked lists:

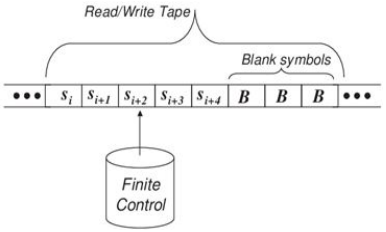
- We have to be very careful regarding where to stop the traversal. For example, if you want to insert at position n, then we have to stop traversal at position n – 1. If you want to read then we stop traversal at n. If we make one more wrong step then we cannot backtrack. You have to restart from the head.

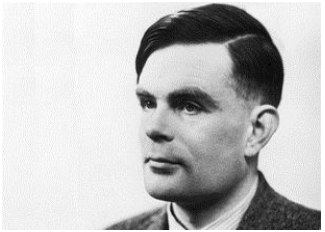
Reasons for doubly linked list





Reasons for doubly linked list





Some of you know about the famous mathematician and computer scientist Alan Turing. He investigated and proved many important properties about computers. One of his most fascinating contributions is the Turing Machine that can simulate any computer. By doing so, the Turing machine allows us to investigate the fundamental limitations and capabilities of all computers ever being built. The machine is surprisingly simple. It has an infinite tape of symbols and a read-write head that can make one-step forward or backward on that tape in each step or change the content of the tape cell where the head is located at that moment. The read-write head, also called the finite control, keeps track of its current state and decides what to do based-on the symbol on the tape in its position and its state.

So what kind of list you need to implement this tape?

Practice

Question 01

Write a function that moves the last node to the front in a given Singly Linked List.

Examples:

Input: 1 → 2 → 3 → 4 → 5
Output: 5 → 1 → 2 → 3 → 4

Input: 3 → 8 → 1 → 5 → 7 → 12
Output: 12 → 3 → 8 → 1 → 5 → 7

```
def move_last_to_front(linked_list):
        if linked_list.head is None or linked_list.head.next is None:
            return

        prev = None
        current = linked_list.head

        # Traverse to the last node and keep track of the previous node
        while current.next:
            prev = current
            current = current.next

        # Set the last node as the new head
        prev.next = None
        current.next = linked_list.head
        linked_list.head = current
```

Question 02

Given two lists sorted in increasing order, create and return a new list representing the intersection of the two lists. The new list should be made with its own memory — the original lists should not be changed.

Example:

Input:
First linked list: 1 → 2 → 3 → 4 → 6
Second linked list be 2 → 4 → 6 → 8,
Output: 2 → 4 → 6.
The elements 2, 4, 6 are common in both the list so they appear in the intersection list.

Input:
First linked list: 1 → 2 → 3 → 4 → 5
Second linked list be 2 → 3 → 4,
Output: 2 → 3 → 4
The elements 2, 3, 4 are common in both the list so they appear in the intersection list.

```
def intersection( list1, list2):
        if list1 is None or list2 is None:
            return None

        intersection_list = LinkedList()
        current1 = list1.head
        current2 = list2.head

        while current1 and current2:
            if current1.data == current2.data:
                intersection_list.add_node(current1.data)
                current1 = current1.next
                current2 = current2.next
            elif current1.data < current2.data:
                current1 = current1.next
            else:
                current2 = current2.next

        return intersection list
```

Insert at last position of linked list, write the function yourself

Question - 06

Given a singly linked list, find the middle of the linked list. If there are even nodes, then there would be two middle nodes, we need to print the second middle element.

Example:
Input: 10 → 20 → 30 → 40 → 50 → None
Ouput: 30

Input: 1 → 2 → 3 → 4 → 5 → 6 → None
Output: 4

Question - 01

Given a non-dummy headed doubly non-circular linked list, write a function that returns true if the given linked list is a palindrome, else false.

Example:
Input: 1 ⇔ 7 ⇔ 7 ⇔ 1 ⇔ None
Output: True

Input: 1 ⇔ 7 ⇔ 4 ⇔ 5 ⇔ None
Output: False

```
def get_last_node(head):
        current = head
        while current.next:
            current = current.next
        return current

def is_palindrome(head):
        left = head
        right = get_last_node(head)

        while left != right and right.next != left:
            if left.data != right.data:
                return False
            left = left.next
            right = right.prev

        return True
```

Question - 02

Given a non-dummy headed doubly non-circular linked list, reverse the list.

Example:
Input: 10 ⇔ 20 ⇔ 30 ⇔ 40 ⇔ 50 ⇔ None
Output: 50 ⇔ 40 ⇔ 30 ⇔ 20 ⇔ 10 ⇔ None

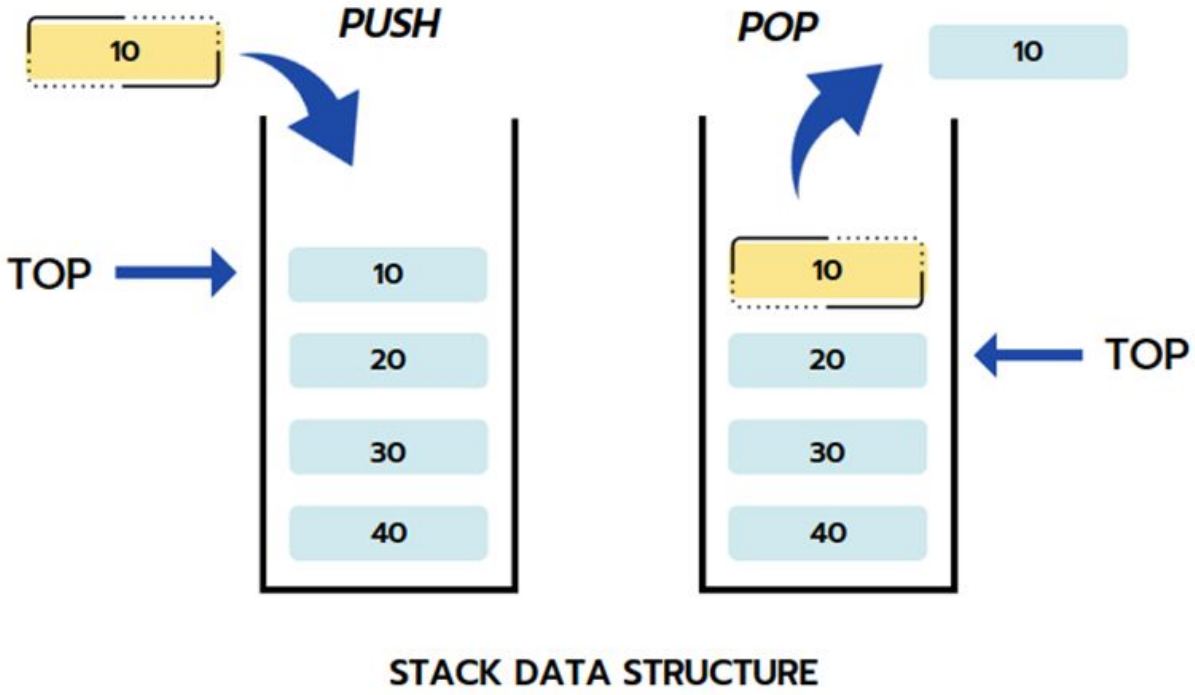
```
def reverse(head):
        current = head
        while current:
            temp = current.prev
            current.prev = current.next
            current.next = temp
            current = current.prev

        head = temp.prev
        return head
```

Next Class: Quiz on linked list

Wednesday 8:00 AM

Stack

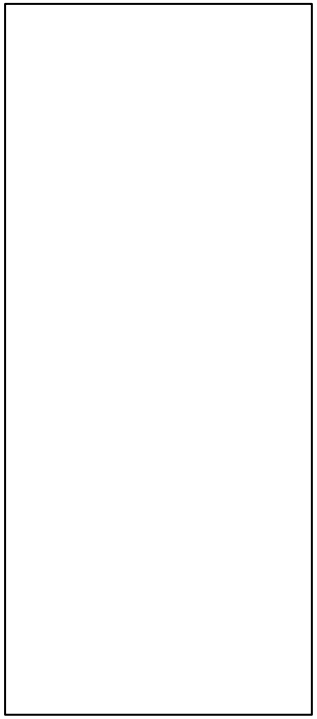
Introduction



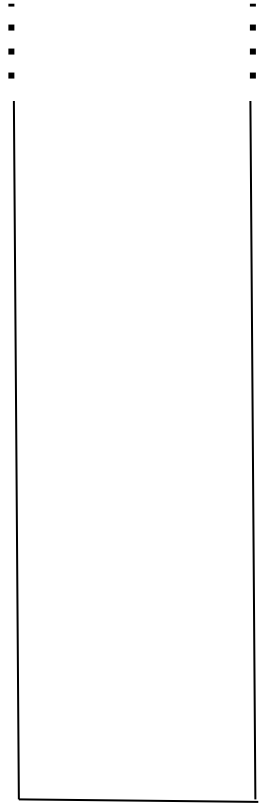
Basic Operations

- **Push(obj):** adds obj to the top of the stack ("overflow" error if the stack has fixed capacity, and is full)
- **Pop:** removes and returns the item from the top of the stack ("underflow" error if the stack is empty)
- **Peek:** returns the item that is on the top of the stack, but does not remove it ("underflow" error if the stack is empty)

Stack



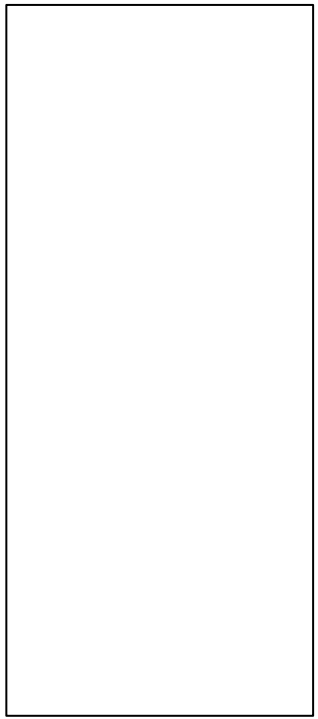
Fixed capacity - **implemented with array**



Unlimited capacity - **implemented with linked list**

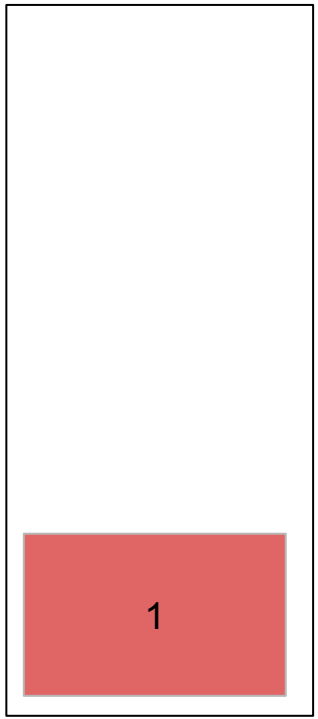
Fixed Capacity Stack Example

Create Stack of Size 4

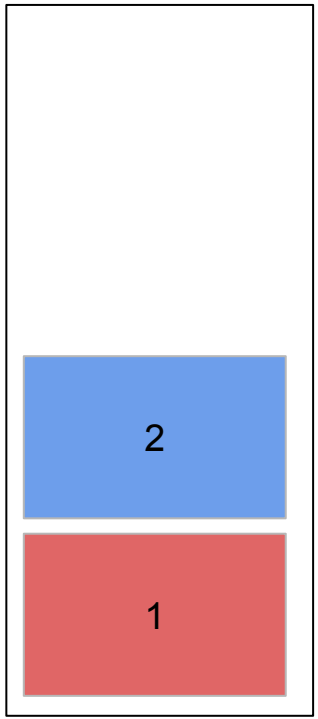


(Implemented with array)

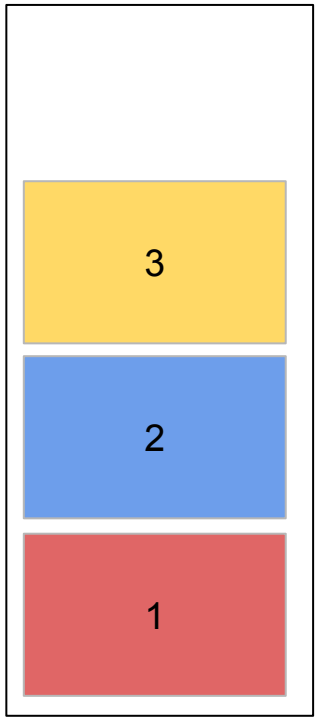
Push(1)



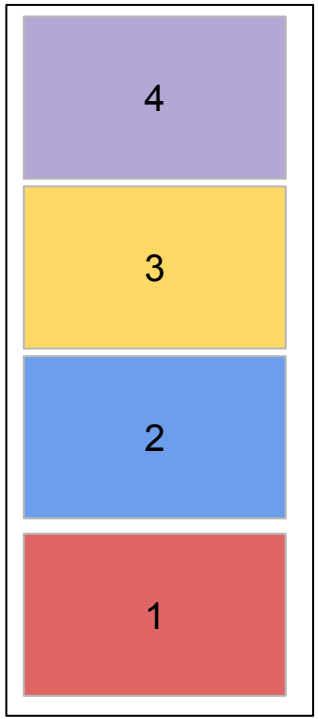
Push(2)



Push(3)

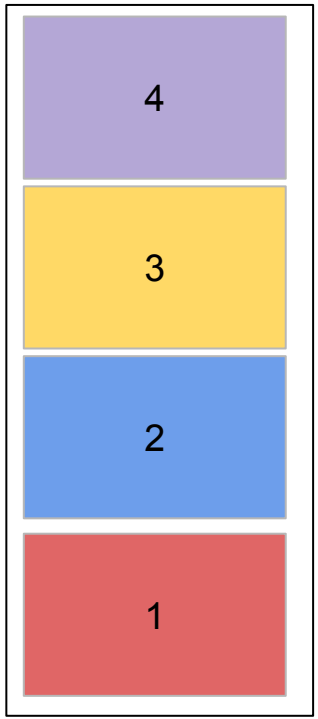


Push(4)

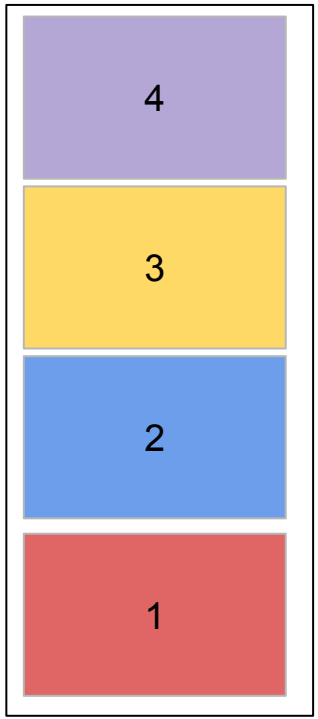


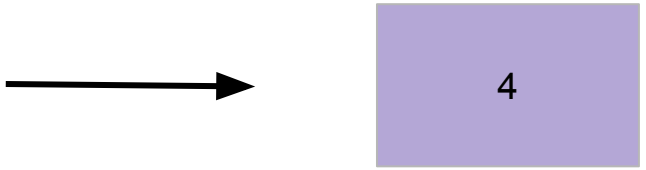
Push(5)

Stack Overflow Error!



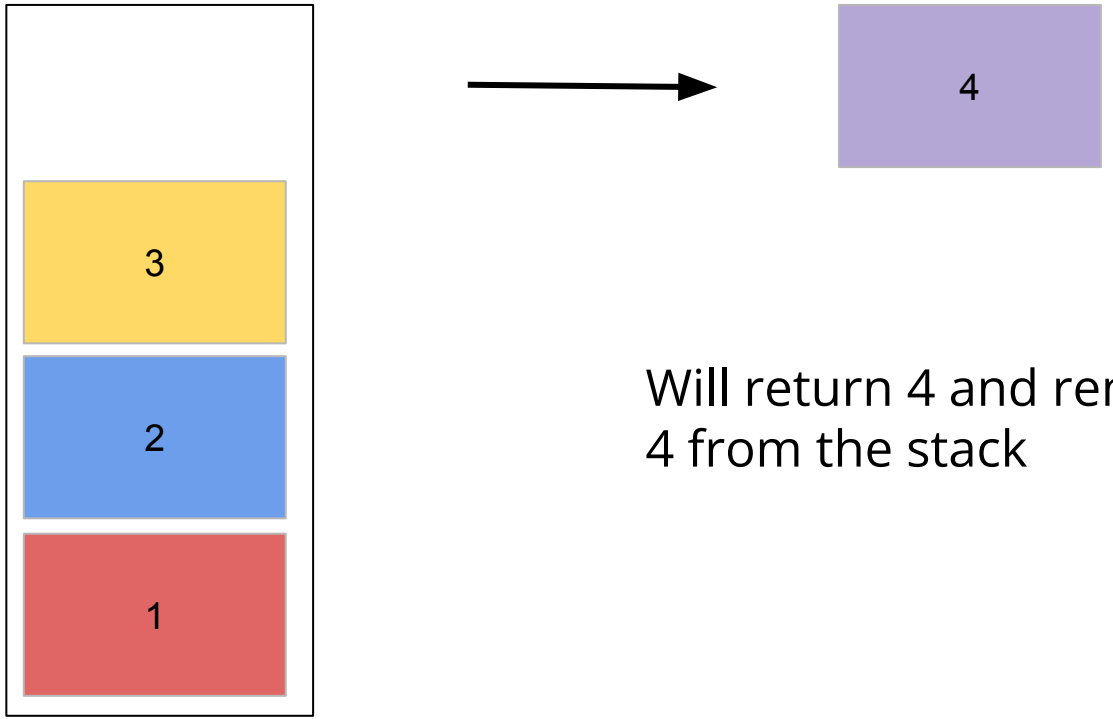
Peek





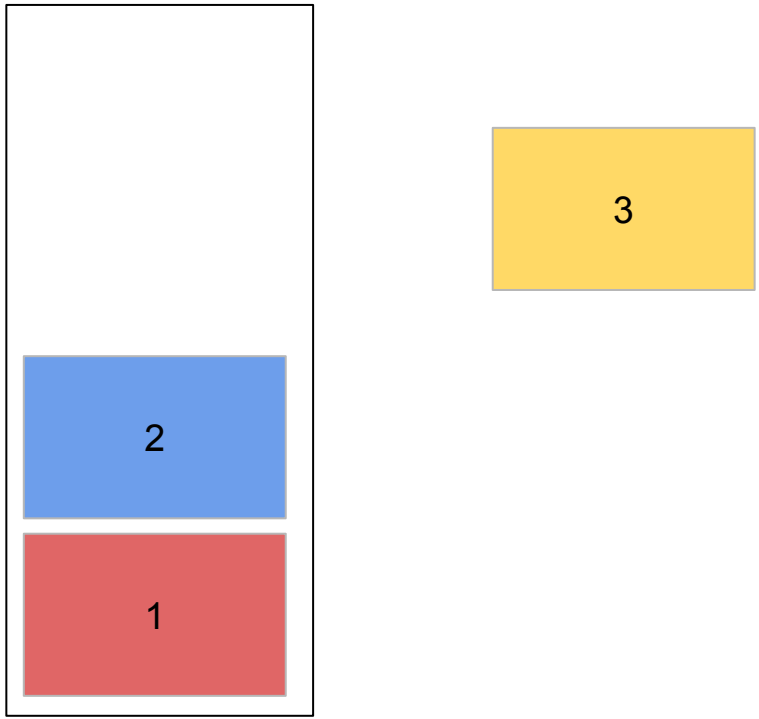
Will return 4, but will not remove 4 from the stack

Pop

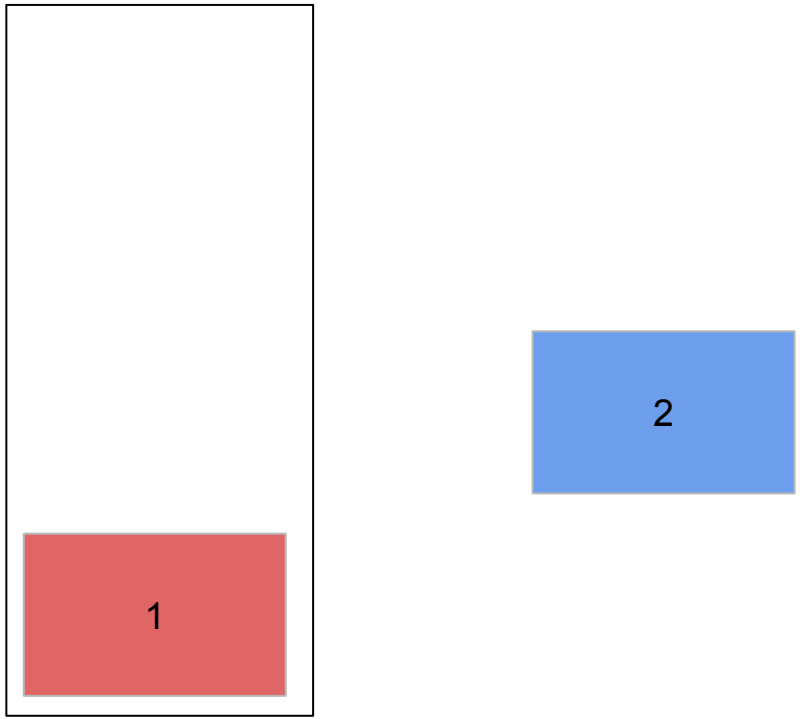


Will return 4 and remove 4 from the stack

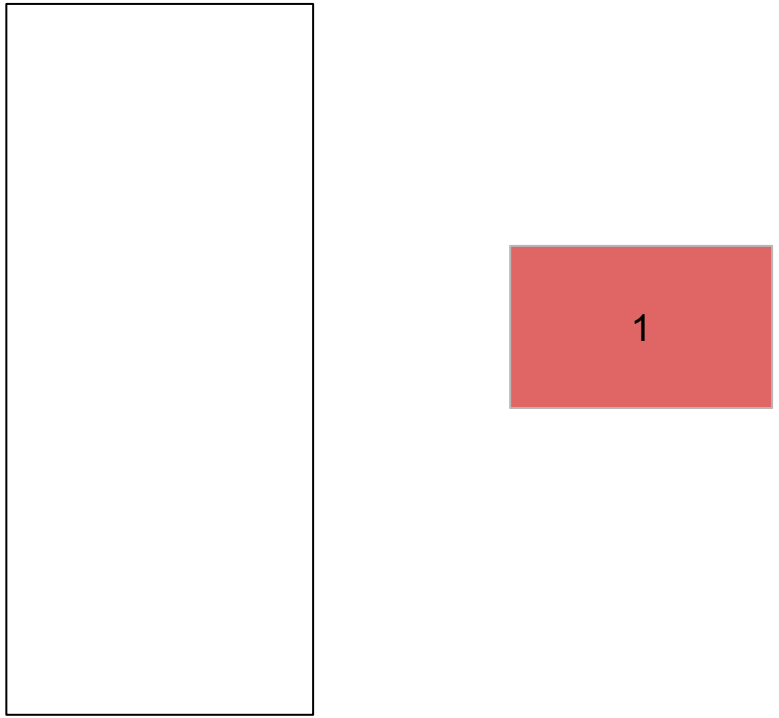
Pop



Pop

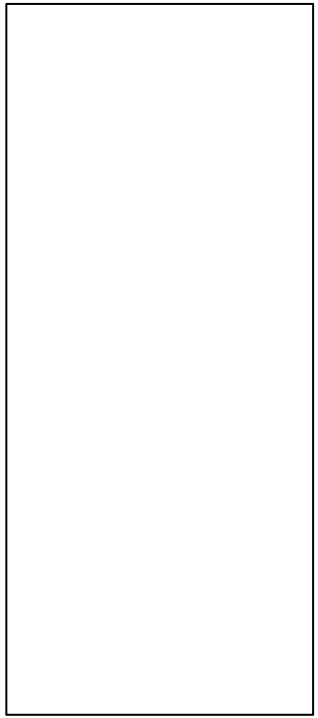


Pop



Pop

Stack Underflow Error!



Code

Implementing The Structure with Array

```
import numpy as np

class Stack:
    def __init__(self):
        self.stack = np.zeros(4, dtype=int)
        self.top = -1

    def push(self, item):
        if self.top == len(self.stack) - 1:
            print("Stack overflow")
        else:
            self.top += 1
            self.stack[self.top] = item

    def pop(self):
        if self.top == -1:
            print("Stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return item
```

```
def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            return self.stack[self.top]
```

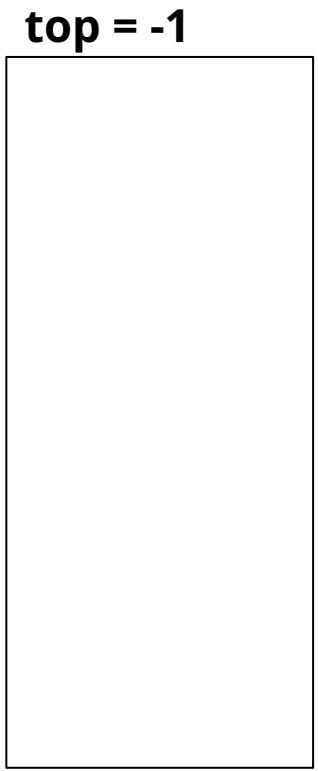
```
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
stack.push(4)
stack.push(5)
print(stack.peek())
print(stack.pop())
stack.pop()
print(stack.pop())
stack.pop()
stack.pop()
```

Driver Code

Using the implemented structure

Fixed Capacity Stack Example

Create Stack of Size 4



Driver code:

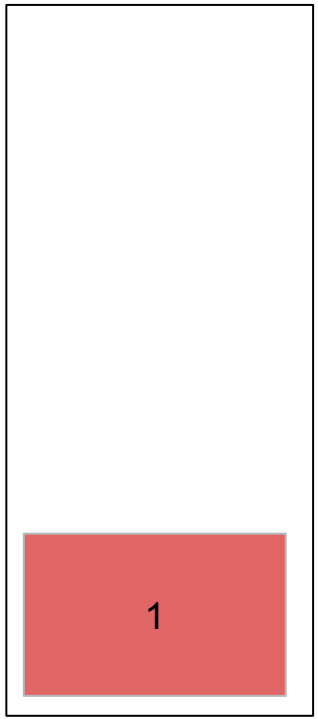
```
stack = Stack()
```

Code running behind the scene:

```
def __init__(self):
        self.stack = np.zeros(4, dtype=int)
        self.top = -1
```

Push(1)

top = 0



Driver code:

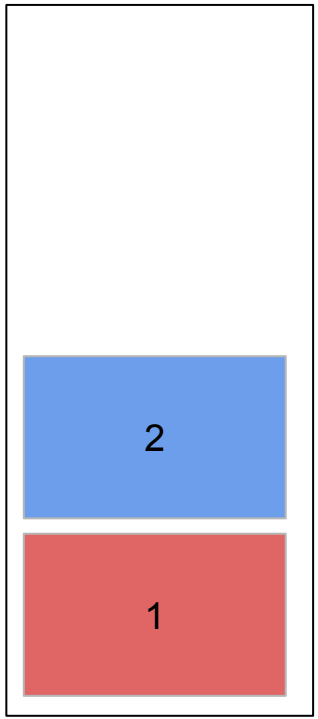
```
stack.push(1)
```

Code running
behind the scene:

```
def push(self, item):
        if self.top == len(self.stack) - 1:
            print("Stack overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
```

Push(2)

top = 1



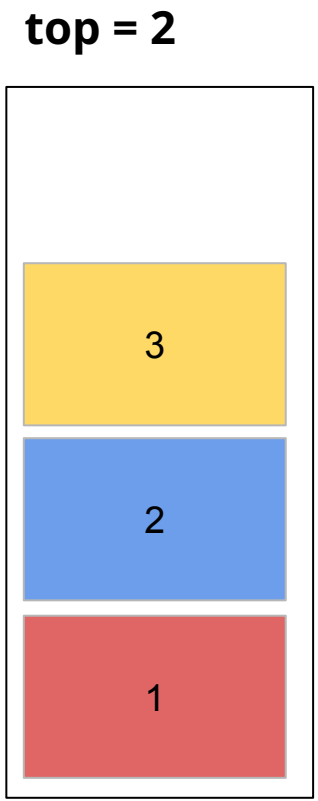
Driver code:

```
stack.push(2)
```

Code running
behind the scene:

```
def push(self, item):
        if self.top == len(self.stack) - 1:
            print("Stack overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
```

Push(3)



Driver code:

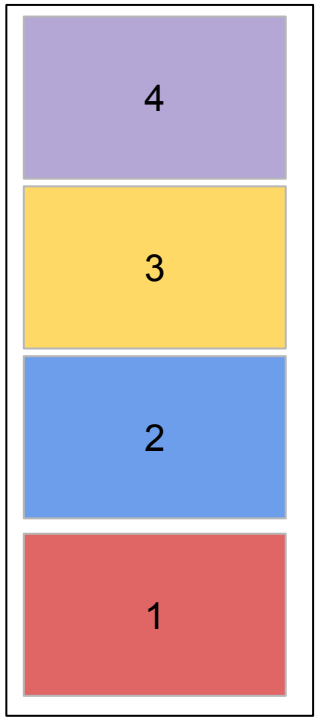
```
stack.push(3)
```

Code running
behind the scene:

```
def push(self, item):
        if self.top == len(self.stack) - 1:
            print("Stack overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
```

Push(4)

top = 3



Driver code:

```
stack.push(4)
```

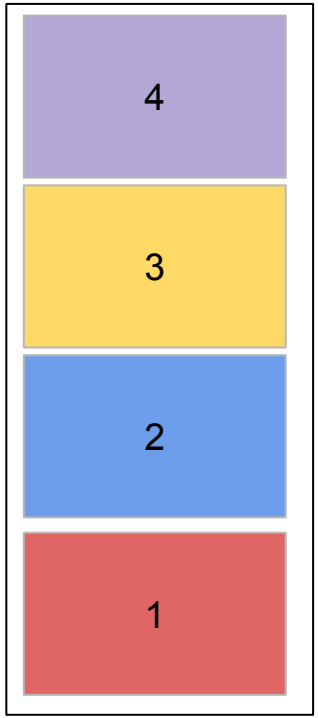
Code running
behind the scene:

```
def push(self, item):
        if self.top == len(self.stack) - 1:
            print("Stack overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
```

Push(5)

Stack Overflow Error!

top = 3



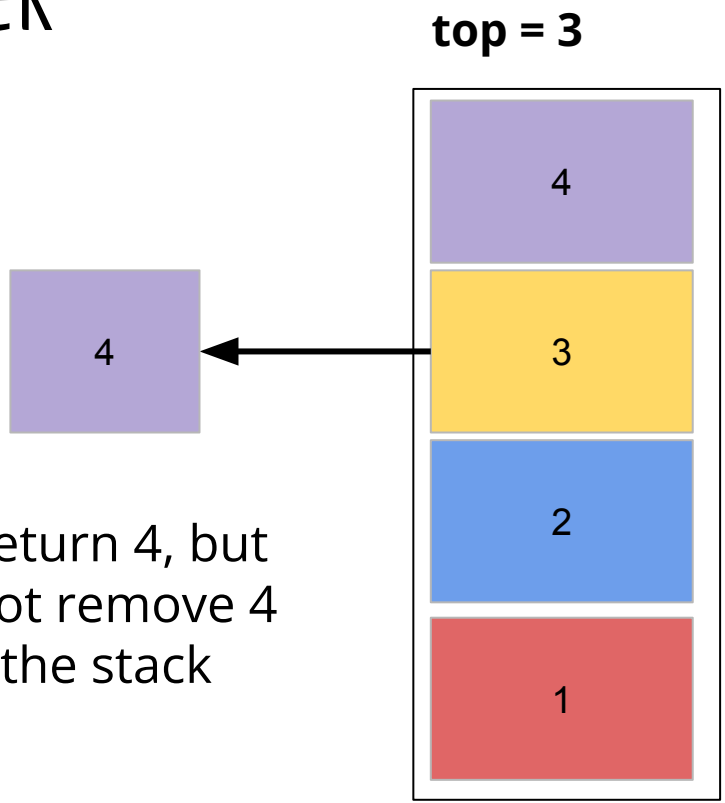
Driver code:

```
stack.push(5)
```

Code running
behind the scene:

```
def push(self, item):
        if self.top == len(self.stack) - 1:
            print("Stack overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
```

Peek



Will return 4, but will not remove 4 from the stack

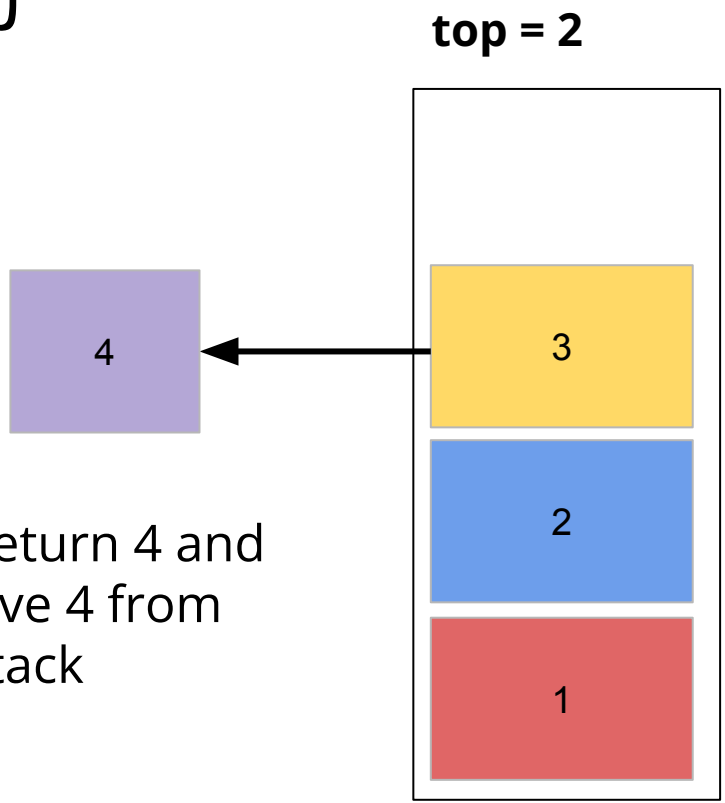
Driver code:

```
print(stack.peek())
```

Code running behind the scene:

```
def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            return
self.stack[self.top]
```

Pop



Will return 4 and remove 4 from the stack

Driver code:

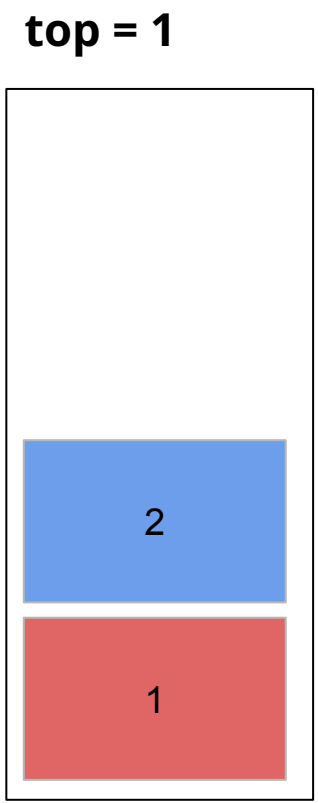
```
print(stack.pop())
```

Code running behind the scene:

```
def pop(self):
        if self.top == -1:
            print("Stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return item
```

Pop

Will remove 3 from the stack



Driver code:

```
stack.pop()
```

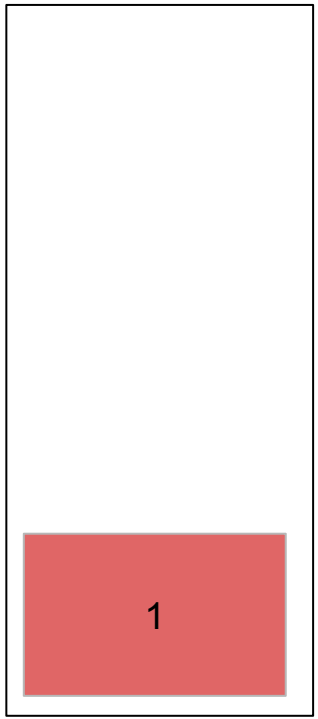
Code running behind the scene:

```
def pop(self):
        if self.top == -1:
            print("Stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return item
```

Pop

Will remove 2 from the stack

top = 0



Driver code:

```
stack.pop()
```

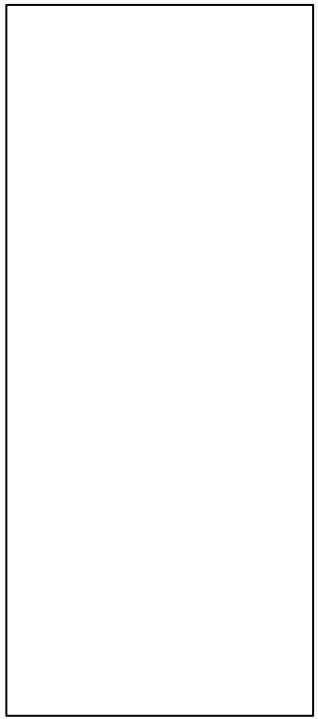
Code running behind the scene:

```
def pop(self):
        if self.top == -1:
            print("Stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return item
```

Pop

Will remove 1 from the stack

top = -1



Driver code:

```
stack.pop()
```

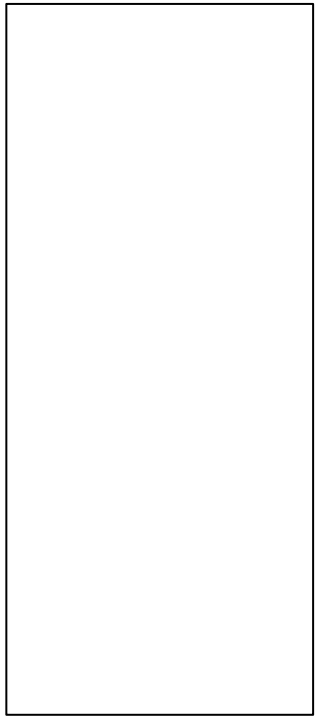
Code running behind the scene:

```
def pop(self):
        if self.top == -1:
            print("Stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return item
```

Pop

Stack Underflow Error!

top = -1



Driver code:

```
stack.pop()
```

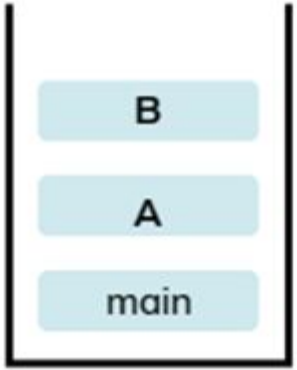
Code running behind the scene:

```
def pop(self):
        if self.top == -1:
            print("Stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return item
```

Stack Applications

1. Call Stacks

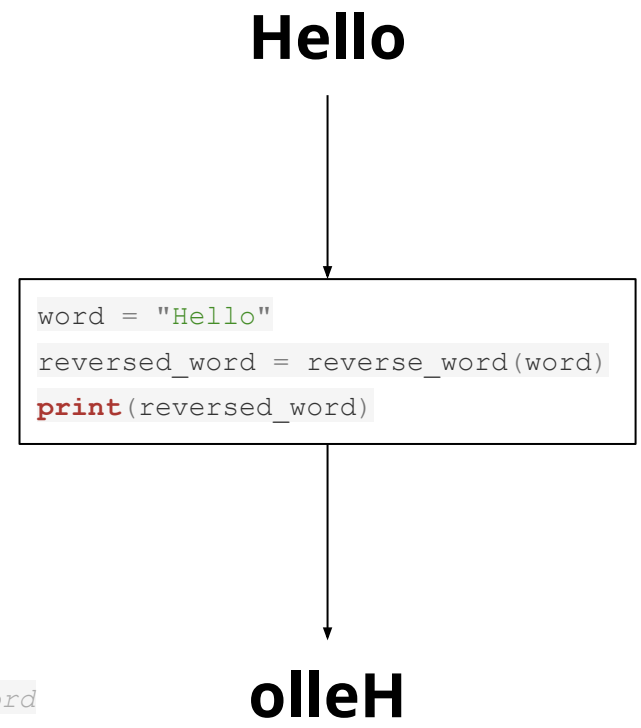
```
def main:
    A()

def A:
    B()

def B:
    print("hello world")

main()
```



Stack Applications

2. Reverse a word

```
def reverse_word(word):
    stack = Stack()

    # Push each character of the word onto the stack
    for char in word:
        stack.push(char)

    reversed_word = ""

    # Pop each character from the stack to get the reversed word
    while not stack.is_empty():
        reversed_word += stack.pop()

    return reversed_word
```



Stack Applications

3. Undo
4. Browser back button